Deciding What to Test First

Even this simple app has several pieces. It's easy to feel overwhelmed as we're deciding what to test first. Where do we start?

To drive the first example, ask yourself: what's the core of the project? What's the one thing we agree our API should do? It should faithfully save the expenses we record.

Let's encode the first part of that desired behavior in a spec, and then implement the behavior. Place the following code in spec/acceptance/expense_tracker_api_spec.rb:

04-acceptance-specs/01/expense_tracker/spec/acceptance/expense_tracker_api_spec.rb

```
require 'rack/test'
require 'json'

module ExpenseTracker
  RSpec.describe 'Expense Tracker API' do
    include Rack::Test::Methods

    it 'records submitted expenses' do
      coffee = {
        'payee'  => 'Starbucks',
        'amount' => 5.75,
        'date'   => '2017-06-10'
      }

      post '/expenses', JSON.generate(coffee)
    end
  end
end
```

Note that we can nest RSpec contexts inside modules. In our codebase, we'll enclose both our app and our specs inside the ExpenseTracker module so that we have easy access to all the classes defined by our app.

Use a Precise Data Type to Represent Currency



To simplify our code examples so you can focus on learning RSpec, we're using regular Ruby floating-point numbers to represent expense amounts—even though floating-point arithmetic isn't precise enough to handle money.[4]

On a real project, we'd either use the BigDecimal class built into Ruby or a dedicated currency library like the Money gem.[5,6]

4. https://spin.atomicobject.com/2014/08/14/currency-rounding-errors/
5. https://ruby-doc.org/stdlib-2.4.1/libdoc/bigdecimal/rdoc/BigDecimal.html
6. http://rubymoney.github.io/money/